C# Programming for Indian Students — Chapter 6: Loops

Nov 2, 2026 · @Vikash

6.1 What you will learn

In Chapter 5 your programs learned to choose. In this chapter they learn to repeat. A teacher prints a mark list of 60 students, an ATM shows its menu again after every transaction, and a form asks for your age again until you type a valid one. All of these use a loop: a block of code that runs again and again while a condition stays true. By the end of this chapter you will be able to:

- repeat code a known number of times with `for`;
- repeat code while a condition is true with `while`, and at least once with `do-while`;
- visit every item of a collection with `foreach`;
- leave a loop early with `break` and skip one round with `continue`;
- ask the user again until the input is valid, instead of ending the program;
- put one loop inside another to print tables and star patterns;
- build a Number Guessing Game that uses a loop, input checking and decisions together.

How input programs are shown. When a program asks the user to type something, the output shows the typed value after the prompt, as it appears on screen. Every program in this chapter was compiled and run on the .NET 10 SDK with the sample input shown.

Remember: all programs use top-level statements, as in Chapters 3 to 5. Each one is a list of lines you place in `Program.cs` of a new console project.

6.2 The for loop

Use a `for` loop when you know how many times to repeat. It keeps the three parts of a counting loop on one line: where to start, when to continue, and how to move to the next round.

```
for (initialization; condition; update)
{
    // body: runs once for every round
}
```

The loop runs in this order:

Vikash Kumar

1. **Initialization** runs once, at the start: `int i = 1`.
2. **Condition** is tested before every round. If it is `false`, the loop ends and the program continues after the closing brace.
3. The **body** runs.
4. **Update** runs after the body (`i++`), and the program goes back to step 2.

The variable declared in the loop header (`i`) exists only inside the loop. Programmers usually name it `i`, `j` or `k`, or something meaningful such as `row` or `year`.

Program 6.1: The for loop

```
// 1. Repeat a message 5 times
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Jai Hind {i}");
}

// 2. Add the numbers 1 to 10
int sum = 0;
for (int number = 1; number <= 10; number++)
{
    sum += number;
}
Console.WriteLine($"Sum of 1 to 10 = {sum}");

// 3. Count down
for (int count = 5; count >= 1; count--)
{
    Console.Write(count + " ");
}
Console.WriteLine("Lift off!");

// 4. Step by 2
for (int even = 2; even <= 10; even += 2)
{
    Console.Write(even + " ");
}
Console.WriteLine();
```

Output

Vikash Kumar

```
Jai Hind 1
Jai Hind 2
Jai Hind 3
Jai Hind 4
Jai Hind 5
Sum of 1 to 10 = 55
5 4 3 2 1 Lift off!
2 4 6 8 10
```

How it works. In the first loop `i` takes the values 1, 2, 3, 4, 5. After the body runs with `i = 5`, `i++` makes it 6, the condition `6 <= 5` is `false` and the loop ends. The second loop is the pattern you will use most: a variable (`sum`) that starts at 0 outside the loop and collects a little in every round. The third loop counts down with `count--`, and the fourth jumps by 2 with `even += 2`. Changing the three parts changes the loop completely.

Remember: `i <= 5` repeats 5 times starting from 1. `i < 5` would repeat only 4 times. Whether to use `<` or `<=` is the most common source of off-by-one mistakes (section 6.7).

Program 6.2: Multiplication table

```
Console.Write("Enter a number: ");
int n = int.Parse(Console.ReadLine() ?? "");

for (int i = 1; i <= 10; i++)
{
    Console.WriteLine($"{n} x {i,2} = {n * i,3}");
}
```

Output (the user typed `7`)

```
Enter a number: 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

How it works. The loop counter `i` is also the multiplier. In `{i,2}` the number after the comma is a field width: the value is printed right-aligned in 2 characters, and `{n * i,3}` uses 3, so the columns line up.

Vikash Kumar

6.3 The while and do-while loops

A `while` loop repeats as long as its condition is true. Use it when you do not know in advance how many rounds are needed, for example "until the number runs out of digits".

```
while (condition)
{
    // body: runs while condition is true
}
```

The condition is tested **before** every round, so if it is `false` at the start, the body never runs. Something inside the body must eventually make the condition `false`; otherwise the loop never ends (an infinite loop, section 6.7).

Program 6.3: The while loop (digits of a number)

```
Console.Write("Enter a positive whole number: ");
int number = int.Parse(Console.ReadLine() ?? "");

int original = number;
int digitCount = 0;
int digitSum = 0;

while (number > 0)
{
    int lastDigit = number % 10;   // take the last digit
    digitSum += lastDigit;
    digitCount++;
    number /= 10;                  // remove the last digit
}

Console.WriteLine($"{original} has {digitCount} digits and the sum of its
digits is {digitSum}.");
```

Output (the user typed `4821`)

```
Enter a positive whole number: 4821
4821 has 4 digits and the sum of its digits is 15.
```

How it works. `number % 10` gives the last digit (the remainder after dividing by 10) and `number /= 10` throws it away, because integer division drops the fraction. 4821 becomes 482, then 48, then 4, then 0, and `0 > 0` is `false`. The original value is saved in `original` first, because the loop destroys `number`.

The do-while loop. A `do-while` loop tests its condition **after** the body, so the body always runs at least once. Note the semicolon after the closing bracket.

Vikash Kumar

```
do
{
    // body: runs at least once
} while (condition);
```

This is the natural choice for a menu: the menu must be shown once before you can ask whether to show it again.

Program 6.4: do-while (a simple bank menu)

```
decimal balance = 1000m;
int choice;

do
{
    Console.WriteLine();
    Console.WriteLine("1. Check balance");
    Console.WriteLine("2. Deposit");
    Console.WriteLine("3. Exit");
    Console.Write("Enter your choice: ");
    int.TryParse(Console.ReadLine(), out choice);

    switch (choice)
    {
        case 1:
            Console.WriteLine($"Balance: Rs {balance}");
            break;
        case 2:
            Console.Write("Amount to deposit: ");
            decimal.TryParse(Console.ReadLine(), out decimal amount);
            if (amount > 0)
            {
                balance += amount;
                Console.WriteLine($"Deposited Rs {amount}. New balance: Rs
{balance}");
            }
            else
            {
                Console.WriteLine("Amount must be more than zero.");
            }
            break;
        case 3:
            Console.WriteLine("Thank you. Goodbye!");
            break;
        default:
```

Vikash Kumar

```
            Console.WriteLine("Invalid choice. Try again.");
            break;
    }
} while (choice != 3);
```

Output (the user typed `1`, then `2` and `500`, then `7`, then `1`, then `3`)

```
1. Check balance
2. Deposit
3. Exit
Enter your choice: 1
Balance: Rs 1000

1. Check balance
2. Deposit
3. Exit
Enter your choice: 2
Amount to deposit: 500
Deposited Rs 500. New balance: Rs 1500

1. Check balance
2. Deposit
3. Exit
Enter your choice: 7
Invalid choice. Try again.

1. Check balance
2. Deposit
3. Exit
Enter your choice: 1
Balance: Rs 1500

1. Check balance
2. Deposit
3. Exit
Enter your choice: 3
Thank you. Goodbye!
```

Vikash Kumar

How it works. The menu and the `switch` from Chapter 5 sit inside the loop. `int.TryParse(..., out choice)` stores 0 when the user types something that is not a number, which falls into `default`. The loop ends only when `choice` is 3. The variable `choice` is declared **before** the `do`, because the condition after `while` is outside the braces and could not see a variable declared inside them.

Which loop? Use `for` when you can count the rounds, `while` when you repeat until something changes, and `do-while` when the body must run at least once.

6.4 The foreach loop, break and continue

A `foreach` loop visits every item of a collection, one after another, without a counter. A collection is anything that holds several values: the characters of a string, or a list of values such as the array in the next program. Arrays get a full chapter later (Chapter 8); here you only need to read the one-line list `{ "Delhi", "Mumbai" }` as "these values, in this order".

```
foreach (type name in collection)
{
    // body: runs once for every item; name holds the current item
}
```

Program 6.5: The foreach loop

```
// A foreach loop visits every item of a collection, here the characters of a
string
string word = "Bharat Mata Ki Jai";
int vowels = 0;

foreach (char letter in word)
{
    if ("aeiouAEIOU".Contains(letter))
    {
        vowels++;
    }
}
Console.WriteLine($"\"{word}\" has {vowels} vowels.");

// ... and the items of a list of values (arrays are covered fully in Chapter
8)
string[] cities = { "Delhi", "Mumbai", "Chennai", "Kolkata" };

foreach (string city in cities)
{
    Console.WriteLine($"City: {city}");
}

int[] marks = { 72, 85, 64, 91, 58 };
int total = 0;
foreach (int mark in marks)
{
```

Vikash Kumar

```
    total += mark;
}
Console.WriteLine($"Total = {total}, Average = {total /
(double)marks.Length}");
```

Output

```
"Bharat Mata Ki Jai" has 7 vowels.
City: Delhi
City: Mumbai
City: Chennai
City: Kolkata
Total = 370, Average = 74
```

How it works. In the first loop `letter` holds each of the 18 characters in turn, and `Contains` checks whether it is one of the ten vowel characters. In the second, `city` holds "Delhi", then "Mumbai", and so on. `marks.Length` is the number of items in the list. The cast `(double)marks.Length` makes the division give a decimal answer, as you learned in Chapter 4.

foreach or for? Use `foreach` when you only need to read every item. Use `for` when you need the position number, want to go backwards, or want to change the items. A `foreach` variable is read-only: assigning to it is error CS1656 (section 6.7).

break and continue. Two keywords change the normal flow of any loop.

- `break;` ends the loop immediately. The program continues after the closing brace of the loop.
- `continue;` ends only the current round. The program jumps to the next round (in a `for` loop, it runs the update `i++` and tests the condition again).

Program 6.6: break and continue

```
// break: stop the loop early
for (int number = 51; number <= 100; number++)
{
    if (number % 7 == 0)
    {
        Console.WriteLine($"First multiple of 7 after 50: {number}");
        break;
    }
}

// continue: skip this round and go to the next one
int oddSum = 0;
for (int number = 1; number <= 10; number++)
```

Vikash Kumar

```
{
    if (number % 2 == 0)
    {
        continue;   // even numbers are skipped
    }
    oddSum += number;
}
Console.WriteLine($"Sum of odd numbers from 1 to 10 = {oddSum}");
```

Output

```
First multiple of 7 after 50: 56
Sum of odd numbers from 1 to 10 = 25
```

How it works. The first loop could run up to 100, but it stops at 56, the first number that divides by 7, so the other 44 rounds never happen. In the second loop, every even number reaches `continue`, skips the line `oddSum += number;` and moves on; only 1, 3, 5, 7 and 9 are added (1 + 3 + 5 + 7 + 9 = 25).

Remember: inside nested loops, `break` and `continue` affect only the innermost loop that contains them.

6.5 Ask again until the input is valid

In Chapter 5 (Program 5.6) a wrong input ended the program with a message: "Please run the program again." Real programs do better. They ask again. This needs a loop that repeats until the input is good, and you do not know how many wrong tries the user will make, so a `while (true)` loop with a `break` is the clearest choice. `while (true)` never ends by itself; the `break` is the only exit.

Program 6.7: Ask again until the input is valid

```
int marks;

while (true)
{
    Console.Write("Enter marks (0 to 100): ");
    string text = Console.ReadLine() ?? "";

    if (!int.TryParse(text, out marks))
    {
        Console.WriteLine($"'{text}' is not a whole number. Try again.");
    }
    else if (marks < 0 || marks > 100)
    {
```

Vikash Kumar

```
        Console.WriteLine($"{marks} is outside the range 0 to 100. Try
again.");
    }
    else
    {
        break;   // valid input: leave the loop
    }
}

Console.WriteLine($"Marks accepted: {marks}");
```

Output (the user typed `abc`, then `150`, then `86`)

```
Enter marks (0 to 100): abc
'abc' is not a whole number. Try again.
Enter marks (0 to 100): 150
150 is outside the range 0 to 100. Try again.
Enter marks (0 to 100): 86
Marks accepted: 86
```

How it works. The checks are the same two-step test as Program 5.6: first `int.TryParse` (is it a number?), then the range test with `||`. The difference is the loop around them. Each wrong answer prints a message and the loop goes round again; the right answer reaches `break`. The variable `marks` is declared before the loop so that it is still available after the loop, where it is used. The compiler accepts this because the only way out of `while (true)` is the `break`, which is reached only after `TryParse` has set `marks`.

Remember: this pattern is the standard way to read input in every program from now on. In the mini project (section 6.8) you will use a variation, where the loop `continue`s instead of `break`s after a wrong input.

6.6 Nested loops and patterns

A loop inside another loop is called a nested loop. For every single round of the **outer** loop, the **inner** loop runs completely, from its start to its end. If the outer loop runs 5 times and the inner loop runs 10 times, the inner body runs 5 × 10 = 50 times. Nested loops are used for anything laid out in rows and columns: tables, grids, seating charts, and patterns.

Program 6.8: Nested loops (multiplication grid)

Vikash Kumar

```
Console.WriteLine("Multiplication table, 1 to 5");
Console.WriteLine("---------------------------");

for (int row = 1; row <= 5; row++)
{
    for (int column = 1; column <= 10; column++)
    {
        Console.Write($"{row * column,4}");
    }
    Console.WriteLine();   // new line after each row
}
```

Output

```
Multiplication table, 1 to 5
---------------------------
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
```

How it works. The outer loop chooses the row. The inner loop uses `Console.Write` (no new line) to print the ten products of that row side by side, each in a field of width 4. When the inner loop ends, `Console.WriteLine()` moves to the next line, and the outer loop starts the next row.

Star and number patterns. Printing a pattern is a nested-loop exercise with one rule: **the outer loop counts the rows, and the inner loop prints the items of one row.** Work out, for each row, how many items it needs. In the right triangle, row *r* has *r* stars. In the pyramid, row *r* has `Rows - r` spaces followed by `2 * r - 1` stars.

Program 6.9: Star patterns

```
const int Rows = 5;

Console.WriteLine("Right triangle:");
for (int row = 1; row <= Rows; row++)
{
    for (int star = 1; star <= row; star++)
    {
        Console.Write("* ");
    }
    Console.WriteLine();
}
```

Vikash Kumar

```
Console.WriteLine();
Console.WriteLine("Pyramid:");
for (int row = 1; row <= Rows; row++)
{
    for (int space = 1; space <= Rows - row; space++)
    {
        Console.Write(" ");
    }
    for (int star = 1; star <= 2 * row - 1; star++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}

Console.WriteLine();
Console.WriteLine("Number triangle:");
for (int row = 1; row <= Rows; row++)
{
    for (int number = 1; number <= row; number++)
    {
        Console.Write(number + " ");
    }
    Console.WriteLine();
}
```

Output

```
Right triangle:
*
* *
* * *
* * * *
* * * * *

Pyramid:
    *
   ***
  *****
 *******
*********

Number triangle:
1
```

Vikash Kumar

```
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

How it works. In the right triangle the inner loop runs `row` times, so the stars grow by one every row. In the pyramid, row 1 prints 4 spaces and 1 star, row 2 prints 3 spaces and 3 stars, and row 5 prints no spaces and 9 stars. In the number triangle the inner loop variable is printed instead of a star, which gives 1 2 3 up to the row number. Change `Rows` to 8 and all three patterns grow. That is the reason to keep the size in a `const`.

Remember: use a different name for the variable of every loop in a nest (`row` , `column` , `star`). Reusing a name is error CS0136 (section 6.7).

6.7 Dry run and common errors

Dry run: a loop one round at a time. To understand any loop, make a table with one row for every round. Write down the loop variable, the result of the condition, and the value of every variable after the body. Follow this program, which uses both `continue` and a running total.

Program 6.10: Dry run

```
int total = 0;
for (int i = 1; i <= 4; i++)
{
    if (i == 3)
    {
        continue;
    }
    total += i * 2;
    Console.WriteLine($"i = {i}, total = {total}");
}
Console.WriteLine($"Final total = {total}");
```

Vikash Kumar

Round	i	i <= 4	i == 3	Work done	total	Prints
start	1	True		`total` starts at 0	0	
1	1	True	False	0 + 1 × 2	2	i = 1, total = 2
2	2	True	False	2 + 2 × 2	6	i = 2, total = 6
3	3	True	True	`continue` : skip the rest of the body	6	nothing
4	4	True	False	6 + 4 × 2	14	i = 4, total = 14
end	5	False		the loop ends	14	

Output

```
i = 1, total = 2
i = 2, total = 6
i = 4, total = 14
Final total = 14
```

Notice that round 3 prints nothing, and that `i++` still ran after `continue`, so the next round started with `i = 4`.

Common errors. The messages below are the real ones printed by the .NET 10 SDK.

Mistake	What C# says	Fix
A semicolon after the loop header: `for (int i = 1; i <= 3; i++);`	warning CS0642: Possible mistaken empty statement	Remove the `;`. The loop runs 3 times doing nothing, and the block below it runs only once.
Using the loop variable after the loop: `Console.WriteLine(j);`	error CS0103: The name 'j' does not exist in the current context	Declare the variable before the loop if you need it afterwards.
The same name in an outer and an inner loop	error CS0136: A local or parameter named 'k' cannot be declared in this scope because that name is used in an enclosing local scope	Give each loop its own name.
Changing the `foreach` variable: `city = "Mumbai";`	error CS1656: Cannot assign to 'city' because it is a 'foreach iteration variable'	Use a `for` loop if you must change items.

Vikash Kumar

Mistake	What C# says	Fix
A statement after `break;` in the same block	warning CS0162: Unreachable code detected	Delete it, or move it before the `break`.
Reading past the end of an array: `for (int i = 0; i <= marks.Length; i++)`	Run-time stop: System.IndexOutOfRangeException: Index was outside the bounds of the array.	Use `i < marks.Length`.

The warnings still let the program build, so they are easy to miss. Read them. The last row is an **off-by-one error**: a list of 3 items has positions 0, 1 and 2, and the loop tried position 3. The program printed 72, 85 and 64 correctly and then stopped with the exception.

The infinite loop. A loop whose condition never becomes `false` runs forever and the program seems frozen. It is the loop mistake every beginner makes once.

```
// Wrong: count never changes, so count > 0 is always true
int count = 5;
while (count > 0)
{
    Console.WriteLine(count);
}

// Correct: the body moves count towards the end
int count = 5;
while (count > 0)
{
    Console.WriteLine(count);
    count--;
}
```

If a program is stuck, press **Ctrl + C** in the console window to stop it. Before running any `while` loop, ask: "What line inside the body makes the condition false one day?" A related trap is `continue` in a `while` loop: if the update line comes after the `continue`, the `continue` skips it and the loop never ends. In a `for` loop the update is in the header, so `continue` is safe there.

6.8 Mini project: Number Guessing Game

Problem. The computer thinks of a secret whole number from 1 to 100. The player gets 7 attempts. After each guess the program says "Too low" or "Too high". The player wins by guessing the number, and the program rates the win by the number of attempts used.

Vikash Kumar

Rules (example rules for this project)

Item	Rule
Secret number	A whole number from 1 to 100 chosen by `Random`
Attempts	At most 7. The game ends when the player wins or the attempts are used up
Invalid input	Not a number, or outside 1 to 100: print a message, do not count it as an attempt, ask again
Rating (win only)	1 to 3 attempts: Excellent! 4 to 5: Good job! 6 to 7: You made it, just in time.

New idea: Random. `new Random()` creates a random number generator, and `random.Next(1, 101)` gives a whole number from 1 up to **but not including** 101, so the result is between 1 and 100. In this book the generator is created as `new Random(2026)`. The 2026 is a *seed*: a seed makes the "random" numbers come out in the same order every run, so the output printed here can be repeated exactly. For a game with a different secret number each time, write `new Random()` with empty brackets.

Program 6.11: Number Guessing Game

```
Random random = new Random(2026);
int secret = random.Next(1, 101);   // a whole number from 1 to 100
const int MaxAttempts = 7;

int attempts = 0;
bool won = false;

Console.WriteLine("Number Guessing Game");
Console.WriteLine($"I am thinking of a number from 1 to 100. You have
{MaxAttempts} attempts.");

while (attempts < MaxAttempts && !won)
{
    Console.Write($"Attempt {attempts + 1}: your guess? ");
    string text = Console.ReadLine() ?? "";

    if (!int.TryParse(text, out int guess) || guess < 1 || guess > 100)
    {
        Console.WriteLine("Please type a whole number from 1 to 100. This
does not count as an attempt.");
        continue;
    }
```

Vikash Kumar

```
    attempts++;

    if (guess == secret)
    {
        won = true;
    }
    else if (guess < secret)
    {
        Console.WriteLine("Too low.");
    }
    else
    {
        Console.WriteLine("Too high.");
    }
}

if (won)
{
    string rating = attempts switch
    {
        <= 3 => "Excellent!",
        <= 5 => "Good job!",
        _ => "You made it, just in time."
    };
    Console.WriteLine($"Correct! You found {secret} in {attempts} attempts.
{rating}");
}
else
{
    Console.WriteLine($"Out of attempts. The number was {secret}.");
}
```

Vikash Kumar

Output (the user typed `50`, `abc`, `25`, `12`, `18`, `16`)

```
Number Guessing Game
I am thinking of a number from 1 to 100. You have 7 attempts.
Attempt 1: your guess? 50
Too high.
Attempt 2: your guess? abc
Please type a whole number from 1 to 100. This does not count as an attempt.
Attempt 2: your guess? 25
Too high.
Attempt 3: your guess? 12
Too low.
Attempt 4: your guess? 18
Too high.
Attempt 5: your guess? 16
Correct! You found 16 in 5 attempts. Good job!
```

How it works.

- The loop condition `attempts < MaxAttempts && !won` uses `&&` from Chapter 4: keep playing while attempts remain **and** the player has not won. The loop ends in one of two ways, and the code after it finds out which one by checking `won`.
- A wrong input reaches `continue`, which skips `attempts++` and returns to the top, so the attempt number printed again is still 2. This is the `continue` version of the pattern from section 6.5.
- `out int guess` declares `guess` inside the `TryParse` call. Because `||` stops at the first `true` part, the range test `guess < 1` runs only after `TryParse` has succeeded, so `guess` always has a value when it is used.
- The comparison uses the `if`, `else if`, `else` ladder of Chapter 5, and the rating uses the switch expression with relational patterns from section 5.8.
- With this seed the secret number is 16. The player's guesses 50, 25, 12 show a good strategy: guess the middle of the remaining range each time. That halves the range every attempt, so 7 attempts are enough for any number from 1 to 100 (2 to the power 7 is 128).

Try to extend it. (1) Ask "Play again? (y/n)" and wrap the whole game in a `do-while` loop. (2) Give a hint "Very close!" when the guess is within 3 of the secret. (3) Let the player choose the highest number (50, 100 or 500) before the game starts.

6.9 Practice questions

Try each question on your own first. The answers follow the list. Programs for Questions 6 to 14 are in the `Solutions` folder of the Chapter 6 code.

Section A: Short answer

Vikash Kumar

1. When do you choose a `for` loop, and when a `while` loop?
2. How is a `do-while` loop different from a `while` loop? Give one situation where `do-while` is the better choice.
3. What is the difference between `break` and `continue`?
4. What is an infinite loop? Write one, and say how to stop a program that is stuck in one.
5. When would you use `foreach` instead of `for`?

Section B: Predict the output

6. What does this program print?

```
for (int i = 1; i <= 3; i++)
{
    Console.Write(i * i + " ");
}
```

7. What does this program print?

```
int x = 10;
while (x > 0)
{
    x -= 3;
    Console.Write(x + " ");
}
```

8. What does this program print?

```
for (int i = 1; i <= 5; i++)
{
    if (i == 2)
    {
        continue;
    }
    if (i == 4)
    {
        break;
    }
    Console.Write(i + " ");
}
```

Section C: Write the program

9. Read a number n (0 to 20) and print n factorial (n! = 1 × 2 × … × n).
10. Read a positive whole number, print its reverse, and say whether it is a palindrome.
11. Read a whole number and say whether it is prime.

Vikash Kumar

12. Read n and print the first n terms of the Fibonacci series (0 1 1 2 3 5 ...).
13. Print a year-by-year growth table of a Rs 10,000 fixed deposit at 7% a year for 5 years, with interest added to the amount every year.
14. Print all Armstrong numbers from 1 to 999 (a number equal to the sum of the cubes of its digits, such as 153 = $1^3 + 5^3 + 3^3$).

Answers

1. Use `for` when the number of rounds is known or can be counted (print 10 lines, visit positions 0 to 9). Use `while` when you repeat until something changes and cannot say in advance how many rounds that takes (digits left in a number, until the input is valid). Both test the condition before every round.
2. A `while` loop tests its condition before the body, so the body may never run. A `do-while` loop tests after the body, so the body runs at least once. A menu that must be shown once before asking whether to show it again is the standard example (Program 6.4).
3. `break` ends the whole loop and the program continues after it. `continue` ends only the current round and goes on with the next round.
4. An infinite loop never ends because its condition never becomes `false`, for example `while (true) { }`, or a `while (count > 0)` loop that never changes `count`. Press Ctrl + C in the console window to stop the program.
5. Use `foreach` when you only need to read every item of a collection one by one and do not need positions. Use `for` when you need the position, a different step, a backward run, or want to change items.
6. `1 4 9` The values of `i` are 1, 2 and 3, and the body prints `i * i`: 1, 4, 9.
7. `7 4 1 -2` The condition is tested before each round, and `x` is reduced by 3 first: 10 → 7 (prints 7), 7 → 4, 4 → 1, 1 → -2. After that `-2 > 0` is `false`. Note that -2 is printed, because the test happens at the top, not after each subtraction.
8. `1 3` For `i = 2`, `continue` skips the print. For `i = 3` the number is printed. For `i = 4`, `break` ends the loop, so 5 is never reached.
9. Program Q09_Factorial. A `long` variable starts at 1 and a `for` loop multiplies it by 2, 3, ... n. Using `long` matters: 20! = 2432902008176640000 does not fit in an `int`. Input `5` gives `5! = 120`.
10. Program Q10_ReverseAndPalindrome. A `while` loop builds the reverse with `reversed = reversed * 10 + number % 10` and removes digits with `number /= 10`. Input `12321` gives "Reverse of 12321 is 12321 / It is a palindrome." and `4821` gives "Reverse of 4821 is 1284 / It is not a palindrome."

Vikash Kumar

11. Program Q11_PrimeCheck. Assume the number is prime when it is at least 2, then try divisors from 2 while `divisor * divisor <= n`. One divisor that divides exactly proves it is not prime, so the loop uses `break`. Input `29` gives prime, `91` gives not prime (91 = 7 × 13). Testing only up to the square root is enough, because every factor pair has one member that small.
12. Program Q12_Fibonacci. Keep two variables `first` (0) and `second` (1). In each round print `first`, then move forward: `next = first + second; first = second; second = next;`. Input `10` gives `0 1 1 2 3 5 8 13 21 34`.
13. Program Q13_FixedDeposit. A `decimal` variable `amount` grows in a `for` loop: `interest = amount * 7 / 100; amount += interest;`. `decimal` is used for money, as in Chapter 3. The last row is year 5: interest 917.56, amount 14025.52.
14. Program Q14_ArmstrongNumbers. An outer `for` loop visits 1 to 999 and an inner `while` loop adds the cube of each digit. The output is `1 153 370 371 407`. The numbers 2 to 9 are not in the list, because $2^3 = 8$, not 2.

6.10 MCQs and viva questions

Multiple-choice questions (answers are listed after the questions)

1. How many times does the body of `for (int i = 0; i < 5; i++)` run?
   - (a) 4
   - (b) 6
   - (c) 5
   - (d) 0
2. Which loop always runs its body at least once?
   - (a) `for`
   - (b) `while`
   - (c) `foreach`
   - (d) `do-while`
3. What does `continue` do?
   - (a) ends the loop
   - (b) skips the rest of the current round and starts the next one
   - (c) ends the program
   - (d) restarts the loop from the beginning
4. What is the value of `sum` after this code?

Vikash Kumar

```
int sum = 0;
for (int i = 1; i <= 4; i++)
{
    sum += i;
}
```

- (a) 4
- (b) 10
- (c) 24
- (d) 6

5. Which loop never ends, because nothing inside it can stop it?
   - (a) `for (int i = 0; i < 3; i++) { }`
   - (b) `while (false) { }`
   - (c) `while (true) { }`
   - (d) `do { } while (false);`
6. How many times does `foreach (char c in "India")` run its body?
   - (a) 4
   - (b) 5
   - (c) 6
   - (d) once
7. An outer loop runs 3 times and the inner loop runs 4 times for each of them. How many times does the inner body run in total?
   - (a) 7
   - (b) 3
   - (c) 4
   - (d) 12
8. Which numbers can `random.Next(1, 101)` return?
   - (a) 1 to 101
   - (b) 1 to 100
   - (c) 0 to 100
   - (d) 2 to 100
9. Which keyword ends a loop immediately?
   - (a) `stop`
   - (b) `exit`
   - (c) `break`
   - (d) `continue`

Vikash Kumar

10. What is the value of `i` after `int i = 10; do { i++; } while (i < 5);`?
    - (a) 10
    - (b) 11
    - (c) 5
    - (d) 0

Answers: 1 (c), 2 (d), 3 (b), 4 (b: 1 + 2 + 3 + 4 = 10), 5 (c), 6 (b), 7 (d: 3 × 4 = 12), 8 (b: the upper limit 101 is not included), 9 (c), 10 (b: the body runs once before the test, then `11 < 5` is `false`).

Viva questions (short oral questions, with model answers)

1. *Name the four loops of C#.* `for`, `while`, `do-while` and `foreach`.
2. *What are the three parts in the header of a `for` loop?* Initialization (runs once), condition (tested before every round) and update (runs after every round).
3. *Why is `while (true)` with a `break` used for reading input?* The number of wrong tries is unknown, and the loop ends only when the input is valid.
4. *What is a nested loop? How often does the inner loop run?* A loop inside a loop. The inner loop runs completely for every single round of the outer loop.
5. *What is an off-by-one error?* A loop that runs one time too many or too few, usually from writing `<` instead of `<=`, or the reverse.
6. *Can you change the variable of a `foreach` loop?* No. It is read-only, and assigning to it is error CS1656.
7. *What does `break` do inside a nested loop?* It ends only the innermost loop that contains it.

6.11 Summary and what comes next

- A loop repeats a block while a condition is true. C# has `for`, `while`, `do-while` and `foreach`.
- `for (start; condition; update)` is the loop for counting. The condition is tested before every round, and the loop variable exists only inside the loop.
- `while` repeats until something changes. If the condition is `false` at the start, the body never runs. Something in the body must eventually make it `false`, otherwise the loop is infinite.
- `do-while` tests after the body, so the body runs at least once. It is the natural loop for menus. Remember the semicolon after `while (condition);`.
- `foreach` visits every item of a collection, such as the characters of a string. Its variable is read-only.

Vikash Kumar

- `break` ends a loop, `continue` ends only the current round. Inside nested loops they affect only the innermost loop.
- To read input until it is valid, put the `TryParse` and range checks inside `while (true)` and `break` when the input is good.
- Nested loops give rows and columns. For patterns, the outer loop counts the rows and the inner loop prints one row. Use a different variable name for every loop.
- The compiler catches some loop mistakes: CS0642 (semicolon after the header), CS0103 (variable used outside its loop), CS0136 (same name reused in a nest), CS1656 (assigning to the `foreach` variable), CS0162 (code after `break`). An off-by-one error appears only at run time, as IndexOutOfRangeException.

Next chapter: Chapter 7, Methods, shows how to give a block of code a name and use it again and again. You will write methods with parameters and return values, split the programs of the earlier chapters into small methods, learn overloading, and see why most programs are built from many small methods rather than one long list of lines.

Vikash Kumar